ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CO5085 – Học sâu và ứng dụng trong thị giác máy tính

BÁO CÁO ASSIGNMENT 1

ResNet-18 và DeiT-Tiny trên CIFAR-100
Khởi tạo, fine-tuning và augmentation có đối chứng

Học kỳ 261 – Năm học 2026–2027

Giảng viên: TS. Lê Thành Sách
TS. Nguyễn Quốc Minh
Sinh viên 1: Phạm Duy Anh - 2310139
Sinh viên 2: Nguyễn Phan Tuấn Duy - 2310491

TP. Hồ Chí Minh, Tháng 11 năm 2026

Mục lục

1 Câu hỏi nghiên cứu và nguồn bằng chứng **2**

2 Dataset và protocol chia tập **3**
2.1 Nguồn và lý do chọn . . . 3
2.2 Split và tính toàn vẹn . . . 3

3 EDA dẫn tới quyết định xử lý **4**
3.1 Cân bằng lớp, định dạng và màu . . . 4
3.2 Lớp gần nhau và giới hạn độ phân giải . . . 5
3.3 Can thiệp dữ liệu được kiểm chứng . . . 6
3.4 Audit ảnh trùng . . . 6

4 Kiến trúc, khởi tạo và freeze masks **7**
4.1 ResNet-18 . . . 7
4.2 DeiT-Tiny . . . 8
4.3 Frozen BatchNorm và kiểm chứng invariants . . . 9

5 Huấn luyện, điều phối và checkpoint **9**
5.1 Recipe thực chạy . . . 9
5.2 Accumulation, AMP và scheduler . . . 10
5.3 Checkpoint, resume và tính hoàn tất . . . 11
5.4 Evaluation và đo chi phí . . . 11

6 Kết quả tám cấu hình chính **12**
6.1 Learning curves của hai họ . . . 13
6.2 Benchmark inference . . . 14

7 Đối chứng augmentation cho scratch **15**

8 Phân tích lỗi và lớp khó **16**

9 Giới hạn và kết luận trong phạm vi study **18**

10 Tái lập và kiểm toán **18**

A Hình bổ sung: scratch và ma trận đầy đủ **19**

1 Câu hỏi nghiên cứu và nguồn bằng chứng

Study so sánh một CNN và một Vision Transformer trên cùng CIFAR-100: ResNet-18 và DeiT-Tiny. Câu hỏi chính là pretraining và phạm vi cập nhật tham số ảnh hưởng thế nào tới chất lượng, chi phí và overfitting trong ngân sách cố định. Hai run scratch không augmentation kiểm chứng một quyết định xử lý dữ liệu rút ra sau EDA.

Có tám cấu hình chính: hai kiến trúc nhân bốn chế độ scratch, head-only, partial và full fine-tune; thêm hai đối chứng scratch noaug. Export đã chạy đủ **25 epoch/run**, seed 42, không dùng mặc định 20 epoch trong notebook sạch để mô tả kết quả 25 epoch. Dataset và recipe của study này độc lập với các notebook Food-101 cũ.

Bảng 1: Truy vết yêu cầu từ `a1_co5085.md` và bằng chứng hiện có.

Yêu cầu / bằng chứng	Phần
Lựa chọn dữ liệu, EDA, split và preprocessing	2, 3
CNN và Transformer, scratch và pretrained, ba freeze strategies	4
Huấn luyện, checkpoint và evaluation có kiểm soát	5
Accuracy, macro-F1, chi phí, inference và lỗi	6–8
Đối chứng xử lý dữ liệu và hạn chế	7, 9
Nguồn reproducible và notebook đã chạy	10
Chấp thuận đổi dataset sau A1.1	Chưa có bằng chứng xác nhận
Video	Chưa cung cấp

Full fine-tune có validation accuracy cao nhất cho cả hai họ trong study này; test tương ứng 81,94% và 82,59%. Đó là kết luận theo bảng đã đo, không phải khẳng định full fine-tune luôn tốt nhất.

Khai báo sử dụng AI

AI được sử dụng để hỗ trợ thiết kế giao diện website và trực quan hóa kết quả; nghiên cứu tài liệu, đối chiếu nguồn tham khảo; xây dựng, rà soát và debug mã nguồn; thiết kế cơ chế điều phối huấn luyện song song bằng hai tiến trình trên hai GPU; tổ chức artifact, kiểm tra tính nhất quán và soạn thảo báo cáo LaTeX. Các số đo trong báo cáo được lấy từ artifact và notebook đã chạy, không thay bằng kết quả do AI tự tạo. Hỗ trợ về mã điều phối không đồng nghĩa AI đã thực hiện thêm các lượt huấn luyện mới trong quá trình biên soạn báo cáo. Hai thành viên chịu trách nhiệm về nội dung và kết quả được trình bày; cần tự rà soát nguồn, kiểm tra mã, hiểu phương pháp và có khả năng giải thích, bảo vệ toàn bộ nội dung trước khi nộp. Việc dùng AI không thay thế các yêu cầu tự thực hiện và hiểu bài trong đề. URL video chưa được cung cấp và không được tự bổ sung.

2 Dataset và protocol chia tập

2.1 Nguồn và lý do chọn

CIFAR-100 [2] có ảnh RGB32 × 32,100 nhãn fine và 20 superclass. Target là nhãn *fine 100 lớp*; superclass chỉ phục vụ tổ chức EDA, không dùng làm target. Dataset có official train 50.000 và test 10.000 ảnh. Nguồn chính thức được trích dẫn nhưng không công bố rõ một giấy phép chuẩn cho dataset tại trang được tham chiếu; báo cáo không tự gán MIT/CC hay suy ra license từ thư viện tải dữ liệu.

CIFAR-100 đủ số lớp để quan sát chuyển giao và lỗi giữa lớp gần nhau, quy mô phù hợp ngân sách hai GPU T 4. Hạn chế quan trọng là độ phân giải rất thấp, trong khi pretrained models nhận 224 pixel. Việc chọn dataset đáp ứng mục tiêu so sánh kiến trúc không đồng nghĩa đã có giảng viên phê duyệt việc đổi dataset sau mốc A1.1. Chưa có tài liệu chấp thuận trong nguồn hiện có.

2.2 Split và tính toàn vẹn

Official train được chia phân tầng seed 42 thành 45.000 train và 5.000 validation; official test 10.000 giữ nguyên. Mỗi lớp có 450/50/100 ảnh. Manifest lưu class mapping, chỉ số, hash dữ liệu và nhãn. Hai họ model và hai đối chứng dùng cùng manifest; không chọn augmentation, siêu tham số hay checkpoint bằng test.

Bảng 2: Split đã đăng ký cho A1.

	Train	Validation	Test
Số ảnh	45.000	5.000	10.000
Số lớp fine	100	100	100
Ảnh/lớp	450	50	100

Study fingerprint:

`5872fe326187fc5fd3117ca96362aee2a13f38d0990aad4d10a90616ff0baa58`

Manifest hash:

`cdd348321726fc1f3159f8e361b69ab5db18c5785e569e000b0ac9982a0617d2`

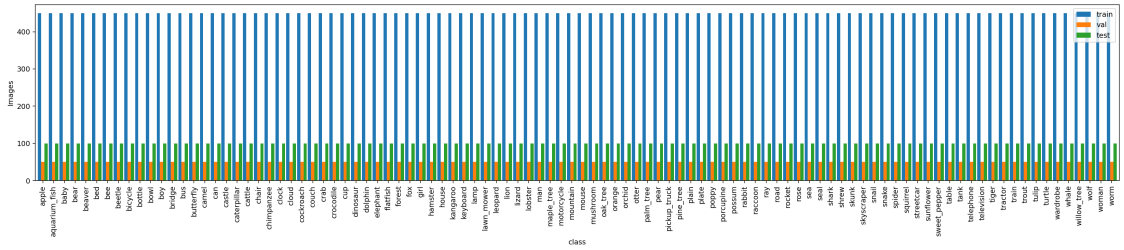
Đây là identity dùng để từ chối ghép các run khác split/recipe hoặc ngân sách. Hash checkpoint được ghi riêng cho từng model; bản bundle nhẹ không chứa checkpoint nên việc kiểm tra lại bundle không được mô tả như vừa đọc lại checkpoint gốc.

3 EDA dẫn tới quyết định xử lý

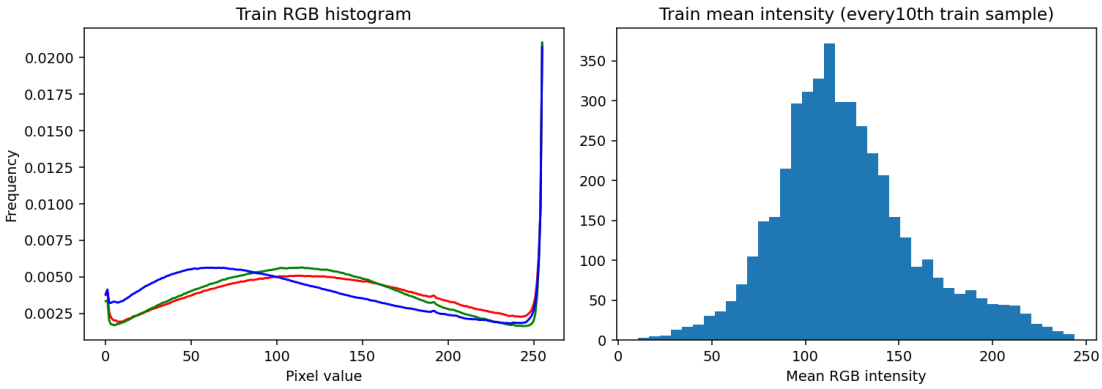
3.1 Cân bằng lớp, định dạng và màu

Audit ghi ảnh shape$[32, 32, 3]$, dtype `uint8`, pixel 0–255; lớp train có min=max 450 ảnh. Vì vậy không dùng class weights hay weighted sampler. Dữ liệu không hợp lệ phải gây lỗi, không âm thầm bỏ mẫu làm thay đổi split.

Mean RGB tính chỉ trên train là $(0{,}507056; 0{,}486423; 0{,}440584)$. Độ lệch chuẩn tương ứng là $(0{,}267242; 0{,}256334; 0{,}276031)$. Thống kê cho thấy các kênh không cùng phân bố. Tuy nhiên pipeline dùng ImageNet normalization chung cho cả hai model, nhằm tương thích checkpoint pre-trained và giữ phép so sánh thống nhất. Không trình bày mean/std CIFAR này như normalization thực dùng.

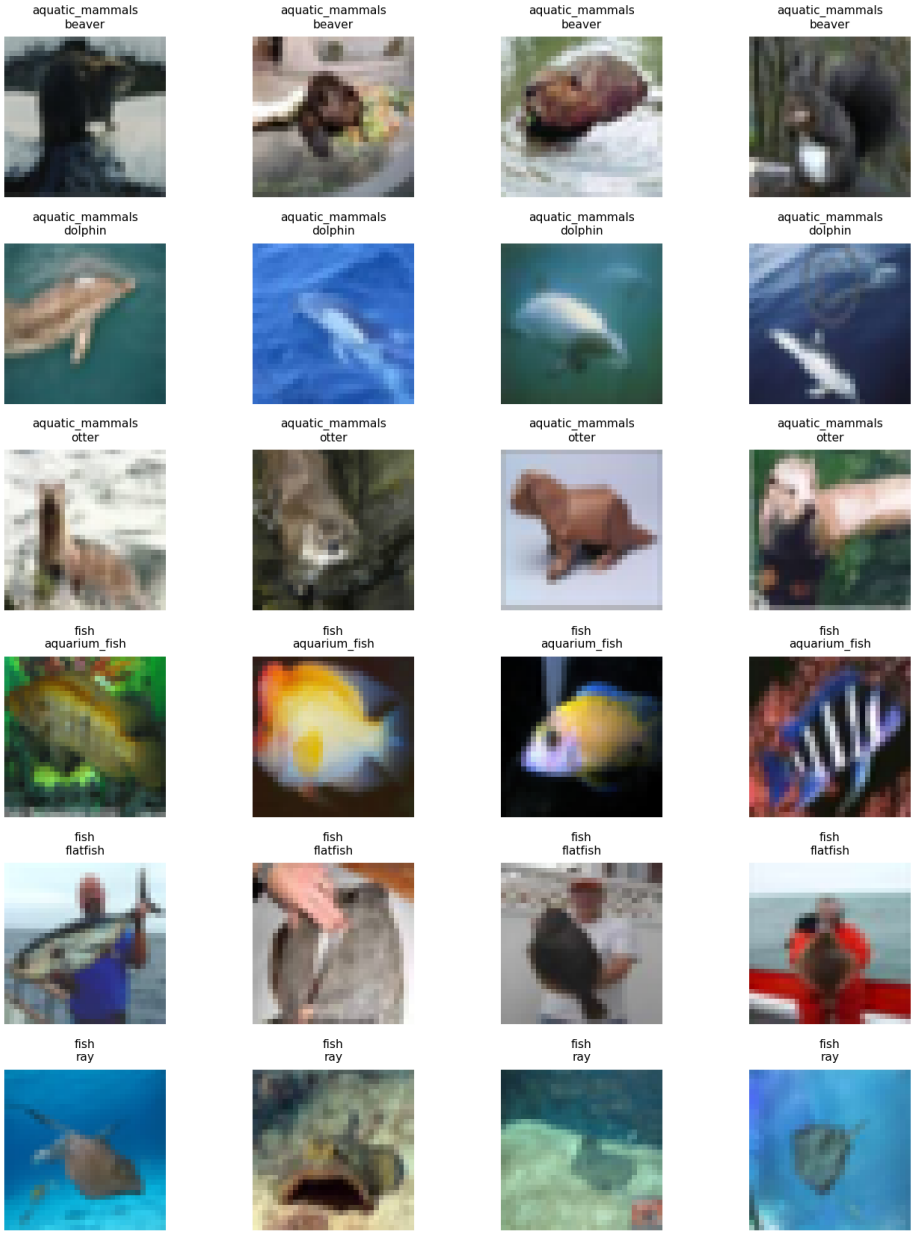


Hình 1: Phân bố số lượng lớp/split từ EDA A1. Quyết định: không bù mất cân bằng lớp.



Hình 2: Phân bố màu và thống kê train. Validation/test không dùng để quyết định transform.

3.2 Lớp gần nhau và giới hạn độ phân giải

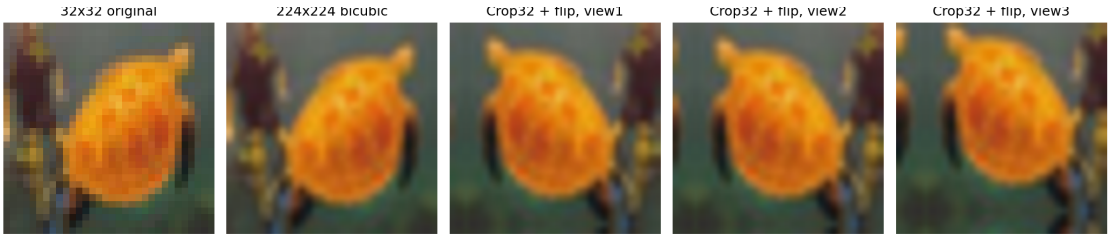


Hình 3: Mẫu train theo các nhóm superclass: hỗ trợ đặt giả thuyết nhầm lẫn, không đổi target 100 lớp.

Nhiều lớp cây, người hoặc thú có bối cảnh và hình dáng gần nhau. Ở 32 pixel, khác biệt lá, khuôn mặt và texture có thể mất. EDA gợi ý xem nhóm lỗi này sau train; việc nhầm thực tế được xác minh bằng top confusion pairs ở Phần 8, không coi trực giác từ ảnh là kết quả model.

Resize 32→224 tăng số pixel đầu vào nhưng không bổ sung thông tin gốc. Dùng chung 224 để tương thích pretrained DeiT patch 16 và ResNet, đổi lại chi phí lớn hơn. Không kết luận đây là độ phân giải tối ưu cho CIFAR-100 vì chưa có ablation input 32/64/224.

3.3 Can thiệp dữ liệu được kiểm chứng



Hình 4: Ảnh gốc, bicubic 224 và ba view augmentation từ cùng ảnh train. Up scale không tạo thêm chi tiết quan sát.

Baseline/evaluation: Resize 224 bicubic, ToTensor, ImageNet Normalize. Pipeline train đề xuất thêm crop và flip *trước* up scale:

```
train_aug = transforms.Compose([
    transforms.RandomCrop(32, padding=4, padding_mode="reflect"),
    transforms.RandomHorizontalFlip(0.5),
    transforms.Resize(224, interpolation=InterpolationMode.BICUBIC),
    transforms.ToTensor(),
    transforms.Normalize([.485, .456, .406], [.229, .224, .225])])
```

Crop nhẹ ở độ phân giải gốc thay đổi vị trí mà vẫn giữ phần lớn nội dung; reflection tránh thêm viền đen mạnh. Flip tăng biến thiên hướng nhưng không chắc hợp mọi mẫu. Đây là giả thuyết cần kiểm chứng, không bảo đảm luôn tăng accuracy. Validation/test dùng baseline xác định; hai run noaug chỉ bỏ crop/flip, giữ seed, split, khởi tạo, optimizer và 25 epoch.

3.4 Audit ảnh trùng

Hash pixel ghi 26 nhóm trùng exact, trong đó 12 nhóm đi qua nhiều split và 17 nhóm có nhãn xung đột. Các số này là số *nhóm*, không phải 26 ảnh hay 12 ảnh. Split đăng ký được giữ để tránh thay đổi đối chứng sau khi thấy kết quả. Báo cáo không khẳng định đã loại leakage; near-duplicate chưa được audit và overlap với pretraining ImageNet chưa được xác minh.

Các lớp như aquarium_fish và nhiều nhóm cây có thể xuất hiện nhiều lần hoặc có nhãn khó phân biệt; một phần nhầm lẫn không chỉ do model. Cần một study mới với chính sách dedup được đăng ký trước để định lượng tác động. Không được trừ một số ảnh sau train rồi dùng test score đó như cùng protocol ban đầu.

Nguồn: `artifacts/A1/duplicate_audit.json`.

4 Kiến trúc, khởi tạo và freeze masks

4.1 ResNet-18

ResNet-18 [1] dùng residual blocks: $y = x + F(x)$ khi cùng shape, hoặc projection shortcut khi đổi stride/kênh. Global average pooling tạo vector 512; classifier `fc` đổi thành Linear512 $\rightarrow$ 100. ModelID `resnet18.a1_in1k` xác định kiến trúc và metadata checkpoint trong timm 1.0.22 [4], không tự nạp weights nếu `pretrained=False`.

```
seed_all(seed)
model = timm.create_model(
    'resnet18.a1_in1k', pretrained=False,
    num_classes=100, drop_rate=0., drop_path_rate=0.)
if strategy != 'scratch':
    timm.models.load_pretrained(model, pretrained_cfg={
        **model.pretrained_cfg, 'file': pretrained_path},
        num_classes=100)
for name, p in model.named_parameters():
    p.requires_grad = (
        strategy in ('scratch', 'full') or name.startswith('fc.')
        or (strategy == 'partial' and name.startswith('layer4.')))
```

`seed_all` cố định RNG trước khởi tạo. Ở scratch, không gọi load pretrained, nên toàn bộ convolution, normalization và classifier bắt đầu bằng khởi tạo mới của implementation. Tên `in1k` trong ID không làm run scratch dùng tri thức ImageNet. Ở ba chế độ còn lại, timm nạp checkpoint ImageNet gốc đã cache/hash; classifier 1000 lớp không tương thích bị bỏ, classifier 100 lớp vẫn mới.

Bảng 3: Phạm vi trainable ResNet-18; tổng 11.227.812 tham số.

Strategy	Module được cập nhật	Số tham số
scratch	Toàn bộ, khởi tạo mới	11.227.812
head-only	`fc`	51.300
partial	`layer4 + fc`	8.445.028
full	Toàn bộ, có pretrained	11.227.812

Mỗi run pretrained nạp độc lập từ checkpoint gốc, không tiếp tục từ model head-only hay partial đã train.

4.2 DeiT-Tiny

DeiT-Tiny [3] đang dùng là `deit_tiny_patch16_224.fb_in1k`. Patch embedding biến 224×224 thành 14×14=196 patch, mỗi embedding rộng 192; thêm class token và position embedding, qua 12 block Transformer. Classifier `head` là Linear192 → 100. Đây là biến thể *không distilled*: pipeline không có teacher, distillation token hay loss distillation, dù paper DeiT có nghiên cứu cơ chế đó.

```
seed_all(seed)
model = timm.create_model(
    'deit_tiny_patch16_224.fb_in1k', pretrained=False,
    num_classes=100, drop_rate=0., drop_path_rate=0.)
if strategy != 'scratch':
    timm.models.load_pretrained(model, pretrained_cfg={
        **model.pretrained_cfg, 'file': pretrained_path},
        num_classes=100)
for name, p in model.named_parameters():
    p.requires_grad = (
        strategy in ('scratch', 'full') or name.startswith('head.')
        or (strategy == 'partial' and name.startswith(
            ('blocks.10.', 'blocks.11.', 'norm.'))))
```

Scratch mở toàn bộ trọng số mới, kể cả patch projection, class token và positional embedding. Ba run pretrained lấy backbone ImageNet gốc rồi thay head 100 lớp. Partial chỉ mở hai block cuối (chỉ số 10 và 11), LayerNorm cuối và head; các block trước, patch embedding và token/position parameters đóng băng.

Bảng 4: Phạm vi trainable DeiT-Tiny; tổng 5.543.716 tham số.

Strategy	Module được cập nhật	Số tham số
scratch	Toàn bộ, khởi tạo mới	5.543.716
head-only	`head`	19.300
partial	`blocks.10, blocks.11, norm, head`	909.412
full	Toàn bộ, có pretrained	5.543.716

So sánh ResNet/DeiT không giữ tổng tham số hoặc recipe pretraining bằng nhau; kết quả phản ánh hai model ID cụ thể, không tách riêng tác động convolution so với attention.

4.3 Frozen BatchNorm và kiểm chứng invariants

`requires_grad=False` ngăn gradient của tham số nhưng không tự ngăn BatchNorm cập nhật running mean/variance trong train mode. Trước mỗi epoch, code gọi `model.train()` rồi chuyển các module có toàn bộ tham số frozen sang `eval()`. Vì vậy head-only ResNet giữ backbone/BatchNorm cố định; partial chỉ cho vùng layer4 train. DeiT chủ yếu dùng LayerNorm; dropout/drop-path được cấu hình 0 nhưng chế độ frozen vẫn được xử lý thống nhất.

```
def frozen_eval(model):
    for module in model.modules():
        params = list(module.parameters())
        if params and not any(p.requires_grad for p in params):
            module.eval()

model.train()
frozen_eval(model)
```

Probe snapshot `state_dict`, RNG và training flags; thực hiện một bước SGD nhỏ trên batch thật; assert output shape$[B, 100]$ và frozen weights/buffers giữ nguyên với `rtol=atol=0`. Trong `finally`, probe phục hồi toàn bộ weights, gradient, RNG và modes để không làm thay đổi điểm bắt đầu của run. Đây là kiểm chứng implementation; không phải một run được tính vào 8+2.

5 Huấn luyện, điều phối và checkpoint

5.1 Recipe thực chạy

Recipe dùng AdamW với LR $3 \cdot 10^{-4}$ và weight decay 0,05. Loss là cross-entropy, không label smoothing, mixup hoặc CutMix. Clip gradient norm 1, dropout/drop-path 0. Warmup 2 epoch rồi cosine trên ngân sách 25 epoch. Cùng recipe cho mọi run nhằm giảm nhiễu khi so pretraining; không khẳng định đây là recipe tối ưu riêng cho scratch.

Microbatch 32, accumulation 2 đặt mục tiêu 64 ảnh/update. Hai process độc lập: GPU0 chạy tuần tự queue ResNet, GPU1 chạy queue DeiT, mỗi queue 5 run. Parent không giữ model trên GPU. Mỗi process sở hữu optimizer, scaler, loader và RNG riêng. Seed data order/worker được xác định theo seed/epoch; augmentation được gắn theo sample/epoch để RNG của model không làm thay đổi cặp dữ liệu được so sánh.

5.2 Accumulation, AMP và scheduler

Với microbatch có n_j ảnh, code cộng loss `reduction='sum'`, backward từng batch, sau đó chia gradient cho $N = \sum_j n_j$ trước optimizer step:

$$g = \frac{1}{N} \sum_j \sum_{i=1}^{n_j} \nabla_\theta \ell(x_{ji}, y_{ji}). \tag{1}$$

Cách này xử lý cả microbatch không đều và nhóm cuối thiếu batch. BatchNorm vẫn thống kê theo microbatch 32; accumulation không tương đương hoàn toàn forward batch 64.

```
loss = F.cross_entropy(logits, labels, reduction='sum')
scaler.scale(loss).backward()
# At the end of an accumulation group:
scaler.unscale_(optimizer)
for p in model.parameters():
    if p.grad is not None:
        p.grad.div_(group_items)
nn.utils.clip_grad_norm_(model.parameters(), 1.0)
# Post-step hook records whether GradScaler actually called optimizer.step.
with optimizer.register_step_post_hook(record_update):
    scaler.step(optimizer)
scaler.update()
optimizer.zero_grad(set_to_none=True)
```

Train dùng AMP FP16 trên CUDA; validation và final evaluation FP32, không autocast. GradScaler có thể bỏ optimizer update nếu gradient không hữu hạn. `AdamW.step()` thành công cũng có thể trả `None`, nên không dùng return value để nhận diện skipped update. Post-step hook ghi nhận step thực; mỗi epoch lưu `optimizer_steps` và `skipped_steps`.

```
if train['optimizer_steps'] > 0:
    scheduler.step()
else:
    print('No optimizer update: scheduler unchanged')
```

Scheduler bước theo epoch sau training, chỉ khi epoch có ít nhất một update thật; không bước theo từng microbatch hay từng update thành công. Histories đã lưu có update ở mọi epoch; các run vẫn có một số attempt bị AMP bỏ. Cơ chế này xử lý cảnh báo scheduler-before-optimizer mà không che warning hoặc làm scheduler chạy trước bước tối ưu thực.

5.3 Checkpoint, resume và tính hoàn tất

Best được chọn bằng validation accuracy, hòa dùng loss. `last.pt` lưu model, optimizer, scheduler, scaler, RNG, history, next epoch và trạng thái best; `best.pt` giữ weights được chọn. Atomic save tránh file dở dang khi process dừng. Resume kiểm tra identity trước ghi đè; thay split, augmentation, ngân sách hoặc version phải tạo study khác.

Resume hỗ trợ ranh giới epoch. Nếu dừng giữa epoch, chạy lại epoch chưa hoàn tất từ checkpoint gần nhất, không tuyên bố khôi phục chính xác giữa batch. Namespace gồm protocol/study fingerprint/full/experiment ID. Run chỉ complete sau đủ epoch, evaluation, benchmark và diagnostics; smoke không nhập vào bảng full. Bundle nhẹ chỉ giữ config, history, metrics và hình, không chứa weights/predictions/raw dataset.

5.4 Evaluation và đo chi phí

Sau khi cả hai worker kết thúc và giải phóng GPU, lần lượt nạp best checkpoint lên GPU0 để FP32 benchmark. Batch 1 và 32, warmup 10, đo 50 lần, synchronize đúng device trước/sau phép đo. Throughput là ảnh/giây; latency ghi ms/ảnh, không phải ms/batch. Peak allocated memory đo theo MiB, không phải toàn VRAM mà nvidia-smi hiển thị.

Training time gồm data loading, forward, backward và optimizer, loại validation/checkpoint I/O. Vì hai queue chạy đồng thời và chia sẻ tài nguyên host/I/O, đây không phải tốc độ GPU cô lập. Environment thực: Python 3.13.16, PyTorch 2.11.0+cu130, torchvision 0.26.0+cu130, timm 1.0.22, NumPy 2.1.3, CUDA13.0, TeslaT 4.

Accuracy, macro-F1 và confusion matrix dùng cùng predictions. Với $C = 100$, $F1_{\text{macro}} = C^{-1} \sum_c 2TP_c/(2TP_c + FP_c + FN_c)$, quy ước 0 khi mẫu số 0. Evaluation cuối chạy đủ 45.000 train,5.000 validation,10.000 test bằng transform xác định. Train accuracy cuối này khác accuracy trong history đo trên ảnh augmented khi model đang train.

6 Kết quả tám cấu hình chính

Bảng 5: Tám cấu hình chính A1; R18 = ResNet-18, DeiT = DeiT-Tiny; F1 là macro-F1 test.

ID	Train%	Val%	Test%	F1%	Best
R18_scratch	84.18	66.44	67.28	67.06	24
R18_head_only	67.62	65.04	65.09	64.84	22
R18_partial	97.56	78.14	77.66	77.64	20
R18_full	99.38	82.08	81.94	81.95	23
DeiT_scratch	77.66	49.88	51.23	50.99	25
DeiT_head_only	75.59	72.56	72.23	72.09	20
DeiT_partial	98.86	76.72	75.44	75.42	22
DeiT_full	99.97	82.66	82.59	82.56	25

Bảng 6: Chi phí training khi hai queue chạy đồng thời; thời gian loại validation/checkpoint I/O.

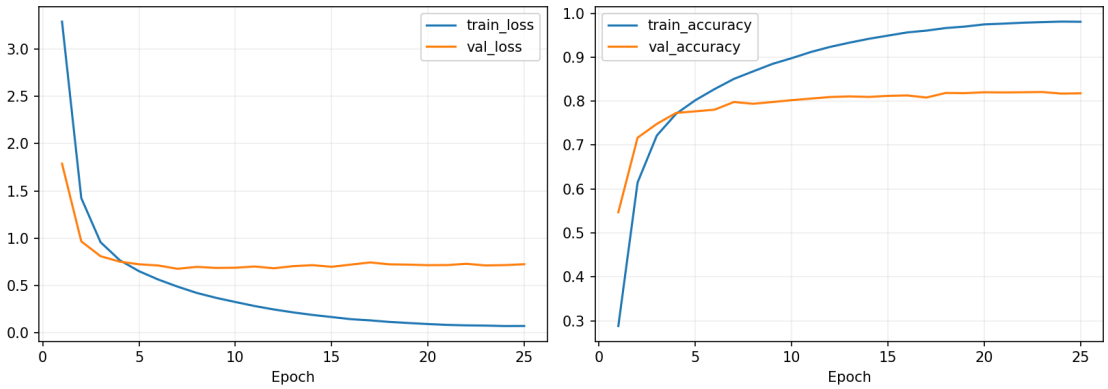
ID	Tổng (M)	Train (M)	Phút train	VRAM (MiB)
R18_scratch	11.228	11.228	65.83	663.3
R18_head_only	11.228	0.051	60.86	248.1
R18_partial	11.228	8.445	62.11	343.1
R18_full	11.228	11.228	62.93	663.3
DeiT_scratch	5.544	5.544	89.77	714.7
DeiT_head_only	5.544	0.019	64.62	141.7
DeiT_partial	5.544	0.909	65.15	226.3
DeiT_full	5.544	5.544	86.01	714.7

Bảng chọn full fine-tune theo validation cho cả hai kiến trúc. DeiT full đạt 82,66% validation và 82,59% test; ResNet full 82,08%/81,94%. Chênh lệch test 0,65 điểm nghiêng về DeiT, song chưa có lặp seed để kết luận khác biệt ổn định. Best epoch trong bảng dùng thứ tự 1–25 đúng như metrics JSON A1; history epoch bắt đầu từ 0 và được đối chiếu riêng.

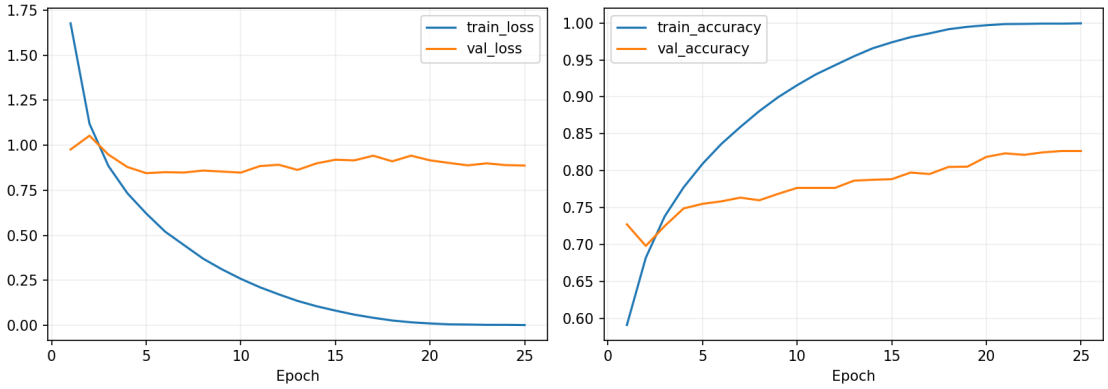
ResNet scratch 67,28% test cao hơn DeiT scratch 51,23% trong recipe cố định 25 epoch. Không suy ra Transformer không học được từ đầu: ngân sách ngắn, LR chung và input up scale có thể bất lợi, còn checkpoint pretrained thay đổi điểm xuất phát mạnh. Head-only DeiT 72,23% cao hơn ResNet 65,09%; partial tăng tới 75,44% và 77,66%. Partial ở hai kiến trúc mở tỷ lệ tham số khác nhau, nên không phải cùng mức compute hoặc capacity thích nghi.

Full cải thiện so với scratch không chỉ phản ánh cập nhật tất cả tham số: cả hai cùng cập nhật toàn bộ, khác chủ yếu khởi tạo. Full so với head-only phản ánh khả năng thích nghi backbone với miền CIFAR. Chi phí training giảm ở head-only vì không backprop toàn bộ backbone; tổng weights và forward vẫn còn, nên không suy ra inference sẽ nhanh tương ứng với số trainable parameters.

6.1 Learning curves của hai họ



Hình 5: ResNet-18 full fine-tune: train augmented và validation xác định.



Hình 6: DeiT-Tiny full fine-tune; không dùng test để chọn mốc tốt nhất.

Khoảng train–validation cần xem cùng loss để nhận diện overfitting. Train accuracy cao không đồng nghĩa checkpoint cuối tốt hơn best. Bảng train cuối đánh giá trên baseline transform, trong khi đường cong train lấy từ quá trình augmentation; không so hai loại train accuracy như cùng một đại lượng.

6.2 Benchmark inference

Bảng 7: Benchmark FP32 tuần tự trên GPU0 Tesla T4, warmup 10, đo 50 lần.

ID	Batch	ms/ảnh	ảnh/s	MiB
R18_scratch	1	4.726	211.6	82.6
R18_scratch	32	0.941	1062.9	303.1
R18_head_only	1	4.999	200.0	82.6
R18_head_only	32	0.974	1026.4	303.1
R18_partial	1	5.099	196.1	82.6
R18_partial	32	1.021	979.2	303.1
R18_full	1	4.797	208.4	82.6
R18_full	32	1.028	973.1	303.1
DeiT_scratch	1	14.651	68.3	56.5
DeiT_scratch	32	1.286	777.4	128.0
DeiT_head_only	1	14.702	68.0	56.5
DeiT_head_only	32	1.310	763.4	128.0
DeiT_partial	1	14.550	68.7	56.5
DeiT_partial	32	1.303	767.7	128.0
DeiT_full	1	15.131	66.1	56.5
DeiT_full	32	1.294	772.8	128.0

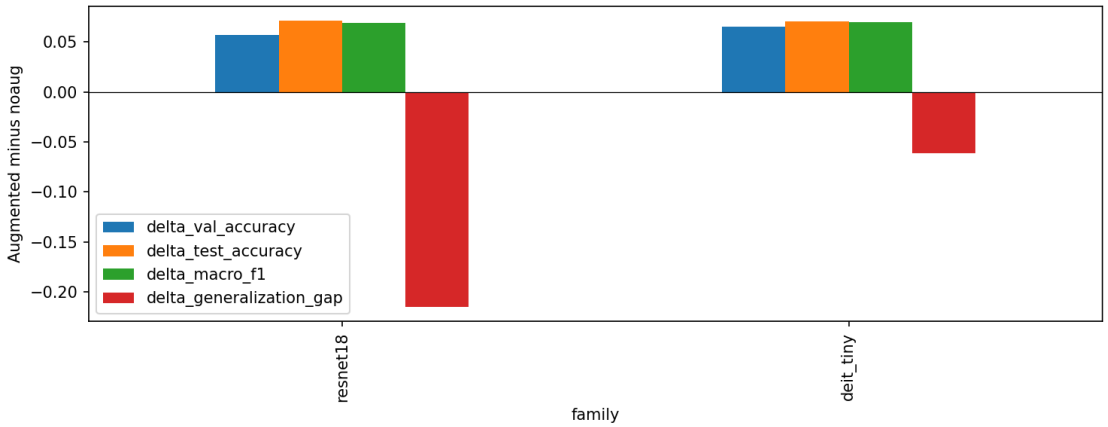
Tất cả số đo ở cùng GPU0 Tesla T4 và FP32, sau khi queues dừng. Batch 32 amortize chi phí dispatch tốt hơn batch 1 nên ms/ảnh thường giảm. Việc freeze chỉ ảnh hưởng training; model khi eval vẫn phải chạy backbone, vì vậy không mong head-only có inference nhỏ tương ứng với 19.300 hoặc 51.300 trainable parameters. Sai khác nhỏ giữa các strategy có thể do dao động đo; không diễn giải như thay đổi kiến trúc.

VRAM training ở bảng trước có optimizer state, gradients, activation và AMP; benchmark memory chỉ đo inference với model được nạp. Hai giá trị không hoán đổi được. Ký hiệu MB trong source dùng chia2^{20} nên báo cáo chuẩn hóa tên thành MiB, giữ nguyên giá trị.

7 Đối chứng augmentation cho scratch

Bảng 8: Đối chứng augmentation. Delta = augmented trừ noaug, tính bằng điểm phần trăm; gap = train trừ validation.

Scratch	Val%	Test%	F1%	Gap (pp)
resnet18 noaug	60.74	60.11	60.13	39.24
resnet18 aug	66.44	67.28	67.06	17.74
resnet18 delta	5.70	7.17	6.93	-21.51
deit_tiny noaug	43.36	44.14	44.03	33.88
deit_tiny aug	49.88	51.23	50.99	27.78
deit_tiny delta	6.52	7.09	6.96	-6.10



Hình 7: Delta augmented–noaug từ hai cặp scratch đã chạy, cùng study.

Test tăng 7,17 điểm phần trăm với ResNet và 7,09 với DeiT. Validation tăng 5,70 và 6,52 điểm. Gap train–validation giảm tương ứng khoảng 21,51 và 6,10 điểm phần trăm. Đặc biệt noaug ResNet có train gần 100% nhưng test 60,11%, cho thấy fit train mạnh mà generalization kém hơn cấu hình aug.

Macro-F1 thay đổi cùng hướng accuracy, hỗ trợ việc cải thiện không chỉ đến từ một vài lớp đông hơn (các lớp vốn cân bằng). Kết luận chỉ áp dụng cho hai cặp scratch đã đo. Không có noaug pretrained trong ma trận này, nên không khẳng định augmentation cải thiện mọi strategy. Giữ cùng seed không ước lượng được biến thiên qua seed; cũng không đủ cho tuyên bố ý nghĩa thống kê.

8 Phân tích lỗi và lớp khó

Bảng 9: Sáu cặp nhầm có hướng đứng đầu mỗi model full, trên test.

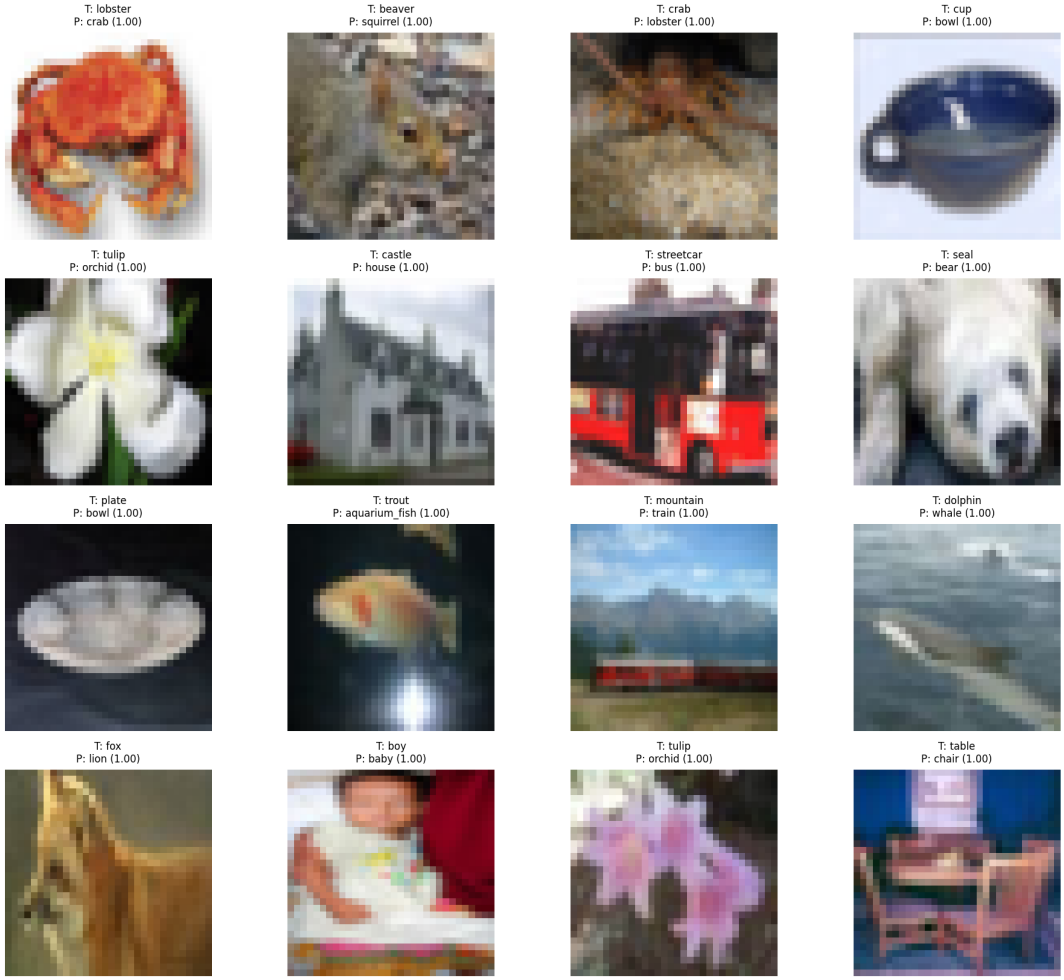
Model	Nhãn thật	Dự đoán	Số ảnh
resnet18	girl	woman	16
resnet18	willow_tree	maple_tree	15
resnet18	seal	otter	15
resnet18	oak_tree	maple_tree	15
resnet18	woman	girl	14
resnet18	boy	girl	14
deit_tiny	oak_tree	maple_tree	19
deit_tiny	girl	woman	18
deit_tiny	man	boy	15
deit_tiny	maple_tree	oak_tree	15
deit_tiny	pine_tree	oak_tree	13
deit_tiny	woman	man	13

Các cặp cây và người xuất hiện trong top nhầm của model tốt nhất, phù hợp giả thuyết EDA về mất texture và chi tiết. Với DeiT full, oak_tree→maple_tree có 19 ảnh; girl→woman có 18 ảnh. Số đếm có hướng: maple→oak không phải cùng ô với oak→maple. Không cộng hai hướng rồi mô tả như một hướng dự đoán.

Bảng 10: Năm lớp có F1 test thấp nhất mỗi model full; support mỗi lớp là 100.

Model	Lớp	Precision%	Recall%	F1%
resnet18	girl	57.61	53.00	55.21
resnet18	otter	53.85	63.00	58.06
resnet18	boy	58.59	58.00	58.29
resnet18	seal	61.05	58.00	59.49
resnet18	shrew	60.38	64.00	62.14
deit_tiny	boy	56.36	62.00	59.05
deit_tiny	girl	66.67	56.00	60.87
deit_tiny	seal	65.26	62.00	63.59
deit_tiny	woman	64.29	63.00	63.64
deit_tiny	pine_tree	68.54	61.00	64.55

Bảng per-class giúp nhìn lớp yếu thay vì chỉ accuracy trung bình. Mỗi lớp test 100 ảnh; thay đổi vài ảnh đã làm recall thay đổi vài điểm phần trăm. Đây là mô tả mẫu test hữu hạn, chưa có interval. Nhãn fine có thể rất gần nhau và duplicate có nhãn xung đột khiến trần chất lượng không rõ; không quy mọi lỗi cho năng lực kiến trúc.



Hình 8: Failure grid DeiT-Tiny full, lấy trực tiếp artifact. Mẫu lỗi minh họa, không đại diện ngẫu nhiên toàn test.

Hình cho phép kiểm tra trực quan nhãn thật, dự đoán và độ tự tin ở các lỗi nổi bật. Confidence cao ở ảnh sai không phải bằng chứng xác suất đã được calibration. Việc chọn nhiều lỗi dễ quan sát không đủ để ước lượng tần suất kiểu lỗi; tần suất phải lấy từ confusion counts hoặc per-class metrics.

Ma trận 100 lớp đầy đủ được đặt ở phụ lục để giữ bố cục chính dễ đọc. Bảng top confusions và per-class ở trên là cách đọc định lượng rõ hơn khi heatmap 100 lớp chỉ hiển thị class ID; bản JSON/CSV vẫn là nguồn tra cứu chính xác.

9 Giới hạn và kết luận trong phạm vi study

Cùng seed/split/recipe và budget giúp so sánh rõ ràng nhưng chưa tạo phép thử tuyệt đối về kiến trúc. ResNet và DeiT khác số tham số, cấu trúc và recipe pretraining. ImageNet có thể chứa nội dung gần CIFAR; chưa audit overlap, nên lợi thế pretrained không được diễn giải như học hoàn toàn trên nguồn độc lập.

12 nhóm duplicate cross-split vẫn được giữ, near-duplicate chưa kiểm tra. Resize 32→224 không tạo chi tiết. Một seed và 25 epoch không chứng minh hội tụ hay ý nghĩa thống kê. Training speed đo khi hai queue đồng thời, không đại diện GPU cô lập. Đổi dataset sau A1.1 cần xử lý theo yêu cầu giảng viên; chưa có bằng chứng đồng ý nên không đánh dấu thủ tục đó hoàn tất.

Trong giới hạn này, full fine-tune được chọn theo validation cho cả hai họ và cho kết quả test cao nhất trong ma trận đã đo. Scratch ResNet thích nghi tốt hơn scratch DeiT ở recipe chung. Augmentation gốc 32 pixel giúp cả hai scratch trong đối chứng, nhưng chưa có dữ liệu cho các strategy khác. Hướng tiếp theo là nhiều seed, budget dài hơn được đăng ký trước, ablation input resolution và study đã dedup; không nhập các đề xuất vào kết quả hiện tại.

10 Tái lập và kiểm toán

Notebook sạch: `A1/a1_cifar100_resnet18_vs_deit.ipynb`. Bằng chứng đã chạy: `A1/resnet18-vs-deit-tiny-on-cifar100.ipynb`. Module worker được tạo bằng các cell `%%writefile` đọc được, hai cell launch queue độc lập; notebook tự chứa khi chạy Kaggle, không cần clone repo.

Artifact báo cáo dùng bundle `artifacts/A1/`; nguồn gốc từ export `A1/A1_full`. Export manifest ghi validation checkpoint và prediction ở lần pack gốc. Lần tạo báo cáo này kiểm tra hash bundle, config, ngân sách và metrics; không huấn luyện lại hoặc đọc checkpoint đã loại khỏi bundle. Công cụ `prepare_reports.py` sinh bảng/hình và provenance, còn `build_reports.py` biên dịch PDF. Không đóng gói weights, predictions hay raw dataset vào LaTeX.

Tài liệu

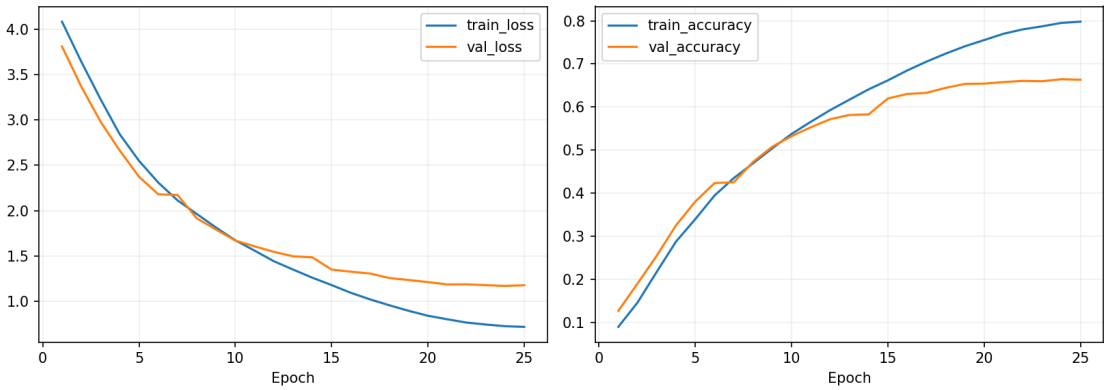
[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. https://arxiv.org/abs/1512.03385, 2015.

[2] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Cifar-10 and cifar-100 datasets. https://www.cs.toronto.edu/~kriz/cifar.html. Nguồn dataset chính thức; truy cập tháng 11/2026.

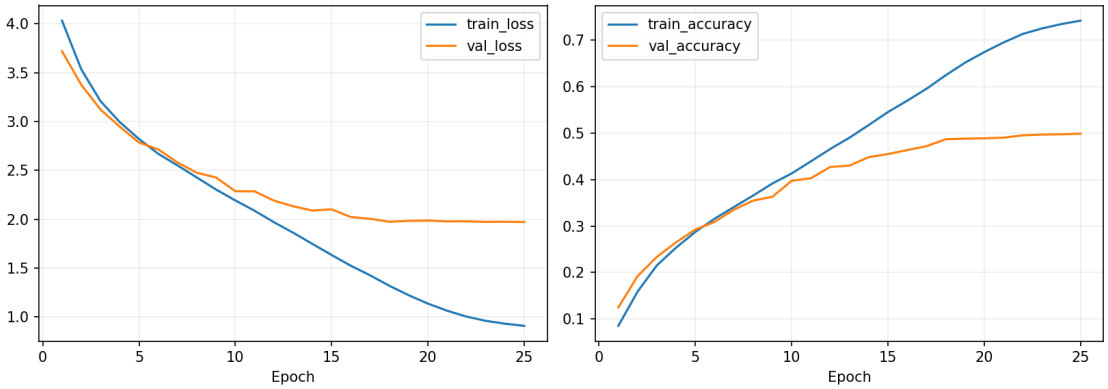
[3] Hugo Touvron et al. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021. https://arxiv.org/abs/2012.12877.

[4] Ross Wightman and contributors. Pytorch image models (timm), version 1.0.22. https://github.com/huggingface/pytorch-image-models/tree/v1.0.22, 2025.

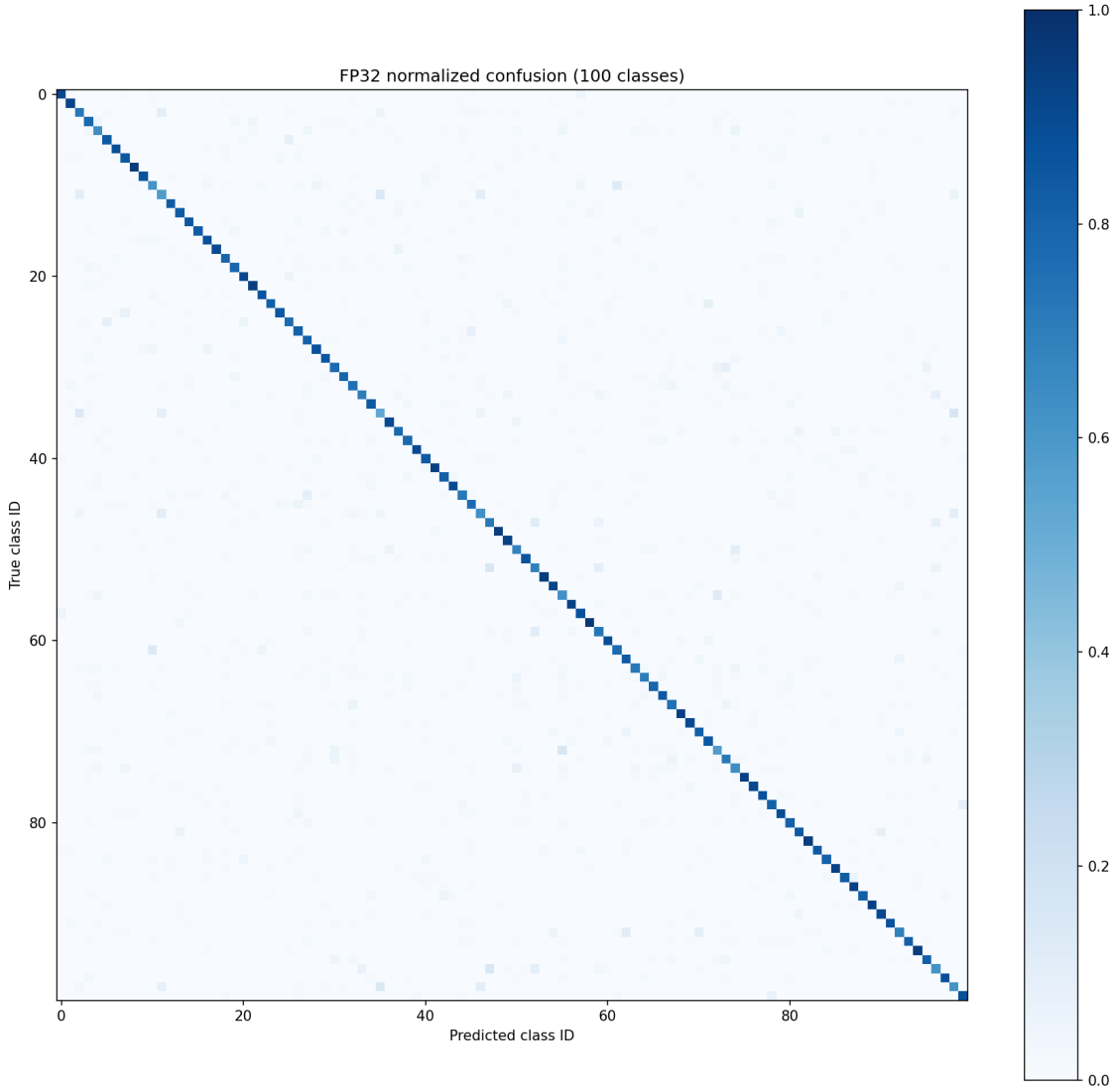
A Hình bổ sung: scratch và ma trận đầy đủ



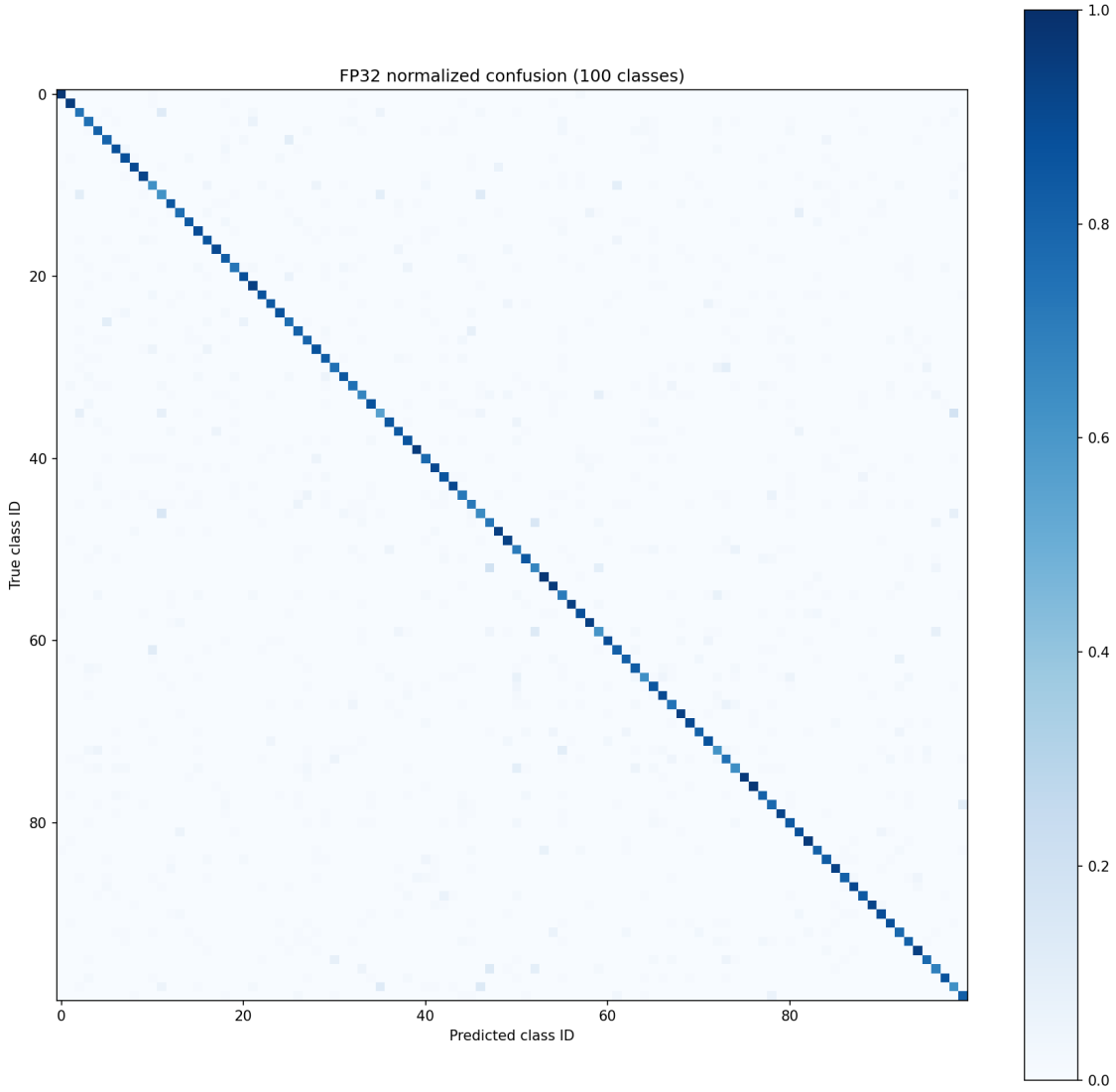
Hình 9: Scratch ResNet với augmentation; đối chiếu noaug ở bảng đối chứng.



Hình 10: Scratch DeiT với cùng ngân sách 25 epoch; không tự tăng epoch riêng.



Hình 11: Confusion matrix 100 lớp ResNet full. Trục dùng class ID; dùng `confusion_matrix.json` để tra ô chính xác.



Hình 12: Confusion matrix 100 lớp DeiT full; bảng top confusions trong phần chính giúp đọc định lượng.